UNIVERSIDAD TECNOLÓGICA DE CHIHUAHUA

Desarrollo y Gestión de Software



9N Desarrollo Web Integral

PRUEBAS UNITARIAS EN SPRING BOOT

PRESENTA:

Diego Emilio Rodríguez Falcón.

Chihuahua, Chih. 11 de noviembre de 2026

1. OBJETIVO DE LA PRÁCTICA

El objetivo principal de mi práctica es comprender y aplicar los conceptos fundamentales de las pruebas unitarias dentro del ecosistema de Spring Boot. A través de este ejercicio, implementare pruebas automatizadas para la capa de servicio de mi aplicación utilizando herramientas como JUnit 5, Mockito y AssertJ. Esto me permitirá validar la lógica de negocio de mi proyecto 'Base_student' de forma aislada, simulando el comportamiento de la base de datos (repositorio) para garantizar que las operaciones de consulta (findAll) y creación (create) funcionen correctamente, con datos de prueba, y de manera confiable antes de llevarlas a un entorno de producción.

2. REQUISITOS PREVIOS Y ENTORNO

- Java Development Kit (JDK) versión 17 o superior instalado y configurado.
- Apache Maven 3.8+ o Gradle como gestor de dependencias.
- Entorno de desarrollo integrado (IDE): IntelliJ IDEA, Eclipse o VS Code con plugins de Java.
- Proyecto base de Spring Boot con Spring Data JPA y una entidad de ejemplo configurada.

3. DESARROLLO DE LA PRÁCTICA

Paso 1: Configuración de Dependencias (pom.xml)

Asegúrate de incluir la dependencia de pruebas en tu archivo *pom.xml*, la cual proporciona JUnit Jupiter, Mockito y AssertJ de manera predeterminada:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

Paso 2: Creación de la Clase de Pruebas Unitarias (*Test.java)

Crea una clase de prueba en el directorio *src/test/java* para evaluar la capa de servicio aislando el repositorio correspondiente mediante Mockito:

Java

```java
package com.base_student.service;

import com.base_student.model.StudentModel;
import com.base_student.model.dto.StudentRsDto;
import com.base_student.repository.StudentRepository;
import com.base_student.service.impl.StudentServiceImpl;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.BDDMockito.given;
import static org.mockito.BDDMockito.then;
import static org.mockito.Mockito.times;

@ExtendWith(MockitoExtension.class)
class StudentServiceTest {

    @Mock
    private StudentRepository studentRepository;

    @InjectMocks
    private StudentServiceImpl studentService;

    @Test
    @DisplayName("Should return all students when findAll is called")
    void shouldReturnAllStudents_WhenFindAllIsCalled() {
        StudentModel student = new StudentModel(1L, "Diego Rodríguez",
"diego@example.com");
        given(studentRepository.findAll()).willReturn(List.of(student));

        List<StudentRsDto> result = studentService.findAll();

        assertThat(result).isNotNull().hasSize(1);
        assertThat(result.get(0).getName()).isEqualTo("Diego Rodríguez");
        then(studentRepository).should(times(1)).findAll();
    }

    @Test
    @DisplayName("Should save and return student when create is called")
    void shouldSaveAndReturnStudent_WhenCreateIsCalled() {
```

```
        StudentModel studentToSave = new StudentModel(null, "Ana López",
"ana@example.com");
        StudentModel savedStudent = new StudentModel(2L, "Ana López",
"ana@example.com");
        given(studentRepository.save(studentToSave)).willReturn(savedStudent);

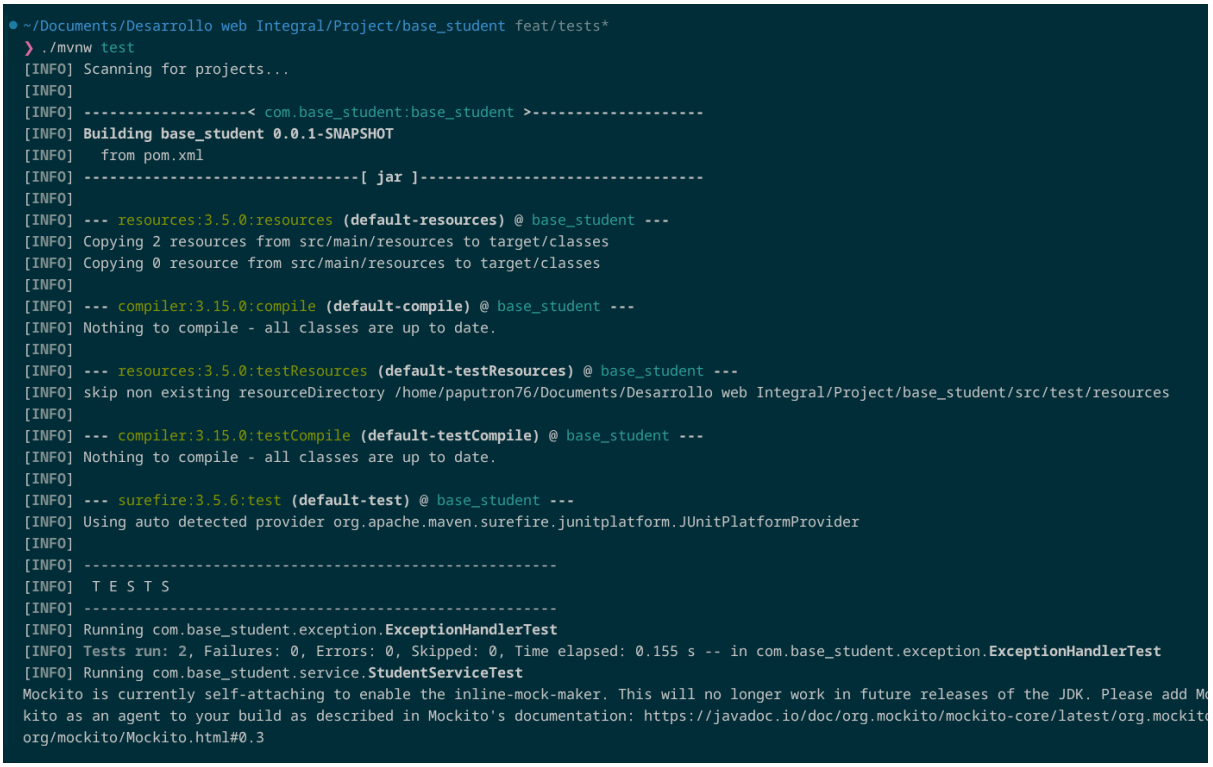
        StudentRsDto result = studentService.create(studentToSave);

        assertThat(result).isNotNull();
        assertThat(result.getId()).isEqualTo(2L);
        assertThat(result.getName()).isEqualTo("Ana López");
        then(studentRepository).should(times(1)).save(studentToSave);
    }
}
```

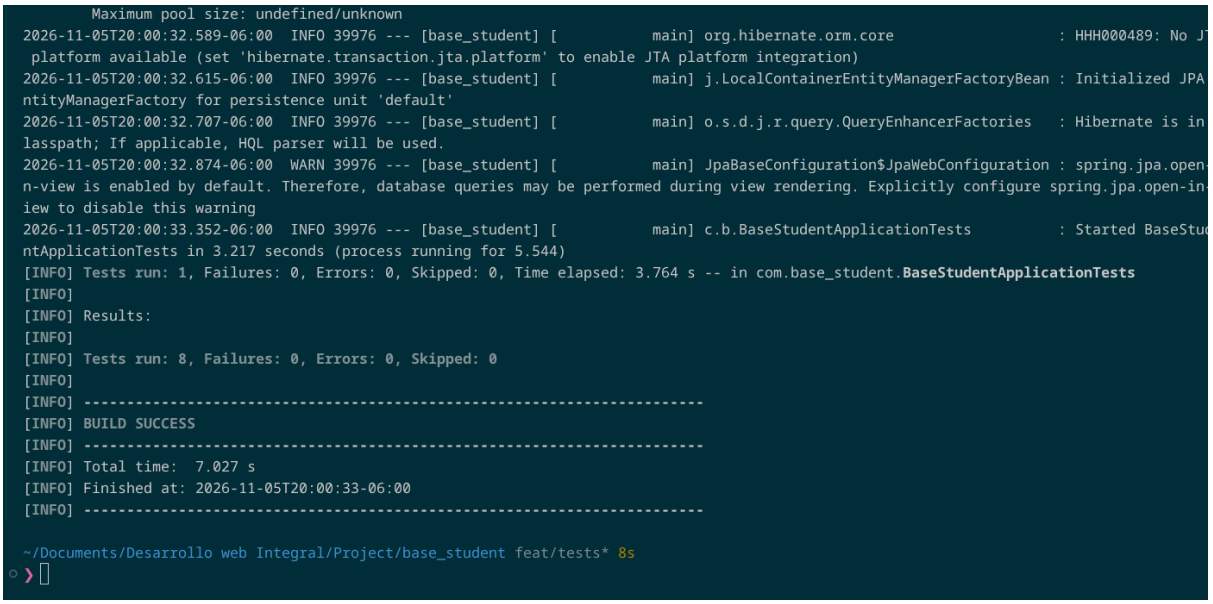
4. Ejecución de la prueba

Para ejecutar la prueba primeramente abrimos una terminal y ejecutamos ./mvn test

al presionar enter comenzará la prueba unitaria que hicimos en StudentServiceTest.java



En este caso la prueba a sido exitosa sin fallos o errores:



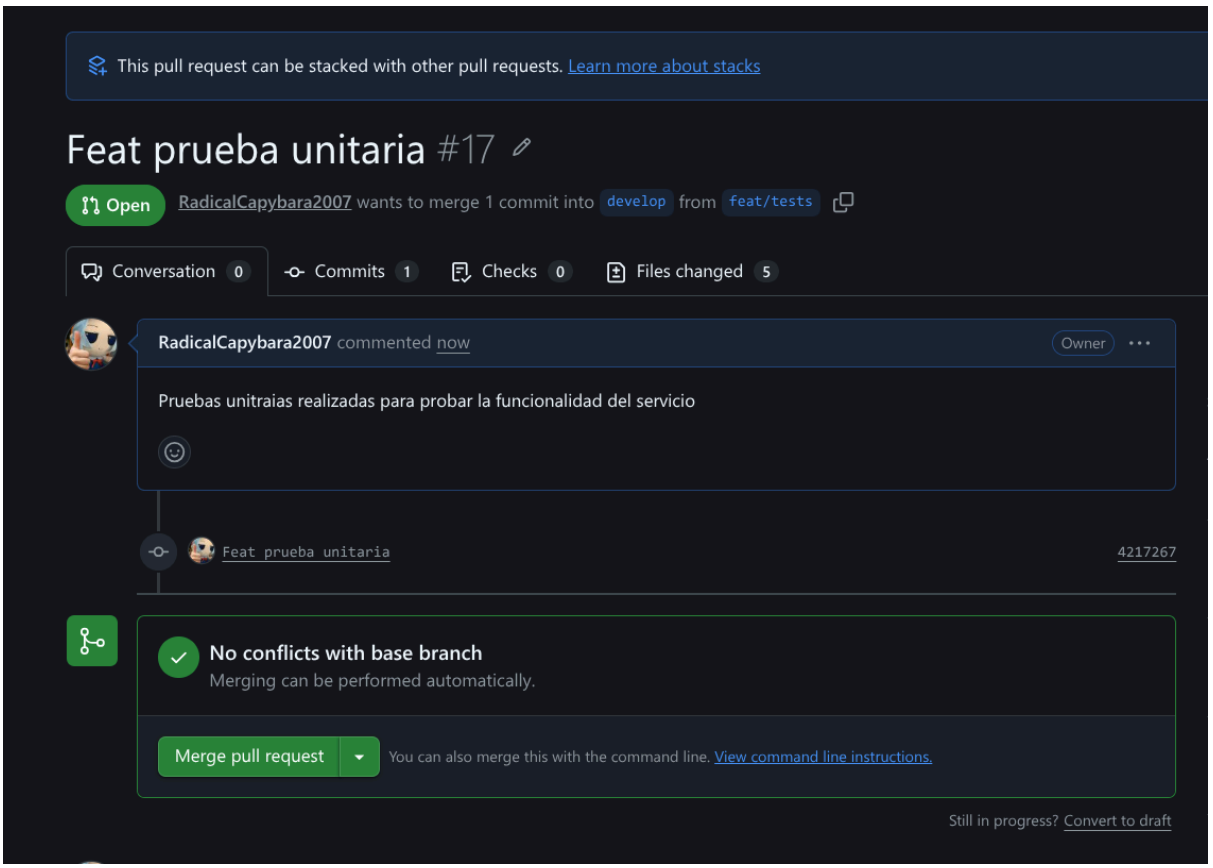
Al finalizar las pruebas se hizo commit y push a la rama remota y de ahí se pasa al cliente web de github para verificar que el commit no provoque fallos y finalmente hacerle merge a la rama develop:

```
~/Documents/Desarrollo web Integral/Project/base_student feat/tests*
❯ git add .

~/Documents/Desarrollo web Integral/Project/base_student feat/tests*
❯ git commit -m "Feat prueba unitaria"
[feat/tests 4217267] Feat prueba unitaria
 5 files changed, 76 insertions(+), 66 deletions(-)
 delete mode 100644 src/test/java/com/base_student/controller/StudentControllerTest.java
 create mode 100644 src/test/java/com/base_student/service/StudentServiceTest.java

~/Documents/Desarrollo web Integral/Project/base_student feat/tests
❯ git push origin feat/tests
Enumerating objects: 32, done.
Counting objects: 100% (32/32), done.
Delta compression using up to 4 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (18/18), 2.03 KiB | 691.00 KiB/s, done.
Total 18 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
remote: This repository moved. Please use the new location:
remote:   https://github.com/RadicalCapybara2007/Base_student.git
remote:
remote: Create a pull request for 'feat/tests' on GitHub by visiting:
remote:      https://github.com/RadicalCapybara2007/Base_student/pull/new/feat/tests
remote:
To https://github.com/RadicalCapybara2007/demo.git
 * [new branch]      feat/tests -> feat/tests

~/Documents/Desarrollo web Integral/Project/base_student feat/tests
❯
```



5. ANEXO: EXPLICACIÓN DETALLADA DEL CÓDIGO DE PRUEBAS

A continuación se detallan los elementos clave implementados en la clase de prueba unitaria *StudentServiceTest*, incluyendo las anotaciones empleadas para aislar las dependencias y el desglose paso a paso de cada caso de prueba según el patrón Arrange-Act-Assert (AAA).

Anotaciones Principales

@ExtendWith(MockitoExtension.class): Habilita la integración de Mockito con JUnit 5. Permite inicializar automáticamente los mocks anotados y gestionar el ciclo de vida de las pruebas sin necesidad de levantar el contexto completo de Spring Boot, logrando una ejecución sumamente rápida y ligera.
@Mock: Crea una instancia simulada (mock) de *StudentRepository*. Con esto evitamos interactuar con la base de datos real, permitiendo definir comportamientos controlados y verificar las interacciones sobre la capa de persistencia.
@InjectMocks: Instancia la clase que se va a probar (*StudentServiceImpl*) e inyecta automáticamente en ella las dependencias marcadas con @Mock (en este caso, el repositorio simulado).

Desglose de Pruebas: Patrón Arrange - Act - Assert - Verify

1. Prueba testFindAll (shouldReturnAllStudents_WhenFindAllIsCalled)

• **Arrange (Preparar):** Se crea una entidad de prueba *StudentModel* con datos ficticios y se define el comportamiento del mock mediante *given(studentRepository.findAll()).willReturn(List.of(student))*, simulando que la base de datos devuelve una lista con dicho estudiante.
• **Act (Actuar):** Se ejecuta el método bajo prueba llamando a *studentService.findAll()* y se almacena el resultado retornado.
• **Assert (Afirmar):** Se valida mediante AssertJ que la lista resultante no sea nula, que tenga un tamaño exacto de 1 y que los datos del estudiante coincidan con los esperados.
• **Verify (Verificar):** Con *then(studentRepository).should(times(1)).findAll()*, se comprueba que el servicio haya llamado al método *findAll()* del repositorio exactamente una vez.

2. Prueba testCreate (shouldSaveAndReturnStudent_WhenCreateIsCalled)

• **Arrange (Preparar):** Se definen dos objetos: uno sin identificador que simula los datos de entrada a registrar, y otro con identificador generado que simula la respuesta persistida por el repositorio mediante *given(studentRepository.save(studentToSave)).willReturn(savedStudent)*.
• **Act (Actuar):** Se invoca el método *studentService.create(studentToSave)* pasando la entidad a guardar.
• **Assert (Afirmar):** Se verifica que el DTO retornado no sea nulo, que contenga el identificador asignado y que los campos coincidan con la entidad guardada.
• **Verify (Verificar):** Se confirma mediante *then(studentRepository).should(times(1)).save(studentToSave)* que la llamada a *save()* fue realizada con el parámetro correcto exactamente una vez.

6. Link al proyecto

https://github.com/RadicalCapybara2007/Base_student.git